



34-MAVZU · 42 TADAN · IV BOB. PYTHON, POSTGRESQL VA DJANGO

Django URL va View komponentlari. URL va View yordamida sahifalar yaratish

So'rov va javob obyektlari, funksiya asosidagi view'lar, URL marshrutlash, path converterlar (int, slug), ilova urls.py va include, nomlangan URL'lar, GET parametrlari, 404 va redirect, Class Based View'lar bilan tanishuv.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- **View** — so'rovni qabul qilib, javob qaytaruvchi funksiya ekanini;
- URL'larni view'larga bog'lash (**marshrutlash**) va ilovaga alohida `urls.py` yaratishni;
- Manzildan qiymat olish: `/maqola/5/` → `id = 5`;
- Qidiruv parametrlari (`?q=...`), 404 xatosi va qayta yo'naltirishni;
- Funksiya va Class asosidagi view'lar farqini.

So'rov va javob

Har bir view — bu **funksiya**: u brauzerdan kelgan **so'rov (request)** obyektni oladi va **javob (response)** obyektni qaytaradi.

```

from django.http import HttpResponse

def salom(request):
    # request – so'rov haqida hamma narsa: metod, manzil, foydalanuvchi,
    # parametrlar...
    print(request.method)          # "GET" yoki "POST"
    print(request.path)            # "/salom/"
    return HttpResponse("Salom!") # javob – brauzerga qaytadigan matn/HTML

```



O'xshatish: pochta oynasi

Pochtaga xat (so'rov) berasiz: unda kimdan, qayerga, nima so'ralgani yozilgan. Xodim (view) xatni o'qiydi, kerakli ishni qiladi va sizga javob xatini (response) qaytaradi. Har bir oyna (URL) o'z xizmatiga mas'ul.

request xossasi	Nima beradi
request.method	So'rov turi: "GET" (sahifani ko'rish) yoki "POST" (forma yuborish)
request.GET	Manzildagi parametrlar: /qidiruv/?q=python → request.GET.get("q")
request.POST	Formadan yuborilgan ma'lumotlar (37-mavzu)
request.user	Tizimga kirgan foydalanuvchi
request.path	So'ralgan manzil

URL marshrutlash

Django har bir so'rov uchun `urlpatterns` ro'yxatini **yuqoridan pastga** ko'rib chiqadi va birinchi mos kelgan yo'lning view'ini chaqiradi.

Ilova uchun alohida urls.py

Barcha URL'larni bitta `config/urls.py` ga yozish katta loyihada chalkashadi. To'g'ri usul: har bir ilova o'z `urls.py` siga ega bo'ladi, asosiy fayl esa ularni `include` bilan ulaydi.

blog/urls.py (yangi fayl yarating)

```
from django.urls import path
from . import views

app_name = "blog" # nomlar maydoni

urlpatterns = [
    path("", views.bosh_sahifa, name="bosh"), # /
    path("haqida/", views.haqida, name="haqida"), #
    /haqida/
    path("maqola/<int:id>/", views.maqola, name="maqola"), #
    /maqola/5/
    path("kategoriya/<slug:nomi>/", views.kategoriya, name="kategoriya"), #
    /kategoriya/sport/
    path("qidiruv/", views.qidiruv, name="qidiruv"), #
    /qidiruv/?q=...
]
```

config/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("blog.urls")), # blog ilovasining barcha URL'lari
]
```

/maqola/5/



config/urls.py

"" → include("blog.urls")



blog/urls.py

"maqola/<int:id>/"



views.maqola(request, id=5)

Path converterlar — manzildan qiymat olish

Converter

Nimaga mos keladi

Misol URL

View oladi

<code><int:id></code>	Musbat butun son	<code>/maqola/42/</code>	<code>id=42</code> (int)
<code><str:nom></code>	<code>/</code> dan boshqa har qanday matn	<code>/user/ali/</code>	<code>nom="ali"</code>
<code><slug:nomi></code>	Lotin harf, raqam, <code>-</code> , <code>_</code>	<code>/kategoriya/maktab-hayoti/</code>	<code>nomi="maktab-hayoti"</code>



Slug — sarlavhaning URL uchun qulay ko'rinishi: "Maktab hayoti" → `maktab-hayoti`. Chiroyli manzillar SEO uchun foydali (11-mavzu).

View'lar yozish

Hozircha ma'lumotlar bazasi o'rniga oddiy Python ro'yxatidan foydalanamiz (36-mavzuda haqiqiy modelga almashtiramiz):

```

from django.http import HttpResponseRedirect, Http404, JsonResponse
from django.shortcuts import redirect

# Vaqtinchalik "baza"
MAQOLALAR = [
    {"id": 1, "sarlavha": "Bilimlar bellashuvi", "kategoriya": "tadbirlar",
    "matn": "8-sinflar o'rtasida..."},
    {"id": 2, "sarlavha": "Yangi kompyuter xonasi", "kategoriya": "maktab-
    hayoti", "matn": "20 ta kompyuter..."},
    {"id": 3, "sarlavha": "Futbol bo'yicha g'alaba", "kategoriya": "sport",
    "matn": "Shahar chempionati..."},
]

def bosh_sahifa(request):
    html = "<h1>Maktab blogi</h1><ul>"
    for m in MAQOLALAR:
        html += f'<li><a href="/maqola/{m["id"]}/">{m["sarlavha"]}</a></li>'
    html += "</ul>"
    return HttpResponseRedirect(html)

def haqida(request):
    return HttpResponseRedirect("<h1>Biz haqimizda</h1><p>5-maktab o'quvchilarining
    blogi.</p>")

def maqola(request, id):
    # id — URL'dagi <int:id> dan
    keladi
    for m in MAQOLALAR:
        if m["id"] == id:
            return HttpResponseRedirect(f"<h1>{m['sarlavha']}</h1><p>{m['matn']}</p><a
            href='/'><← Orqaga</a>")
    raise Http404("Bunday maqola topilmadi") # 404 sahifa

def kategoriya(request, nomi):
    topilganlar = [m for m in MAQOLALAR if m["kategoriya"] == nomi]
    if not topilganlar:
        return redirect("blog:bosh") # bosh sahifaga qayta
    yo'naltirish
    royxat = "".join(f"<li>{m['sarlavha']}</li>" for m in topilganlar)

```

```

return HttpResponse(f"<h1>Kategoriya: {nomi}</h1><ul>{royxat}</ul>")

def qidiruv(request):
    q = request.GET.get("q", "").strip() # /qidiruv/?q=futbol
    natija = [m for m in MAQOLALAR if q.lower() in m["sarlavha"].lower()] if q
    else []
    return HttpResponse(f"<h1>'{q}' bo'yicha {len(natija)} ta natija</h1>")

def api_maqolalar(request):
    return JsonResponse({"maqolalar": MAQOLALAR}) # JSON javob (39-mavzuga muvaddima)

```

Sinab ko'ring:

- `http://127.0.0.1:8000/` — ro'yxat
- `/maqola/2/` — maqola; `/maqola/99/` — 404
- `/kategoriya/sport/`; `/kategoriya/yoq/` — bosh sahifaga qaytaradi
- `/qidiruv/?q=futbol`



Diqqat

View ichida HTML'ni satr sifatida yozish — faqat o'rganish uchun. U noqulay va xavfli (foydalanuvchi matni tekshirilmay HTML'ga qo'shilyapti). Keyingi mavzuda HTML'ni **shablonlarga** (templates) ko'chiramiz — Django ularda matnni avtomatik xavfsiz qiladi.

Nomlangan URL'lar

Har bir yo'lga `name=` berdik. Endi manzilni qo'lda yozish o'rniga nomi bo'yicha yasash mumkin:

Python

```

from django.urls import reverse

reverse("blog:maqola", args=[5]) # "/maqola/5/"
redirect("blog:haqida")          # /haqida/ ga yo'naltirish
# Shablonlarda: {% url 'blog:maqola' 5 %} (35-mavzu)

```

Afzalligi: kelajakda `"maqola/<int:id>/"` ni `"post/<int:id>/"` ga o'zgartirsangiz, faqat `urls.py` da bitta qator o'zgaradi — butun saytdagi havolalar o'zi yangilanadi.

HTTP status kodlari Django'da

Holat	Django'da	Kod
Oddiy javob	<code>HttpResponse(...)</code>	200
Topilmadi	<code>raise Http404(...)</code> yoki <code>get_object_or_404</code> (36-mavzu)	404
Qayta yo'naltirish	<code>redirect("nom")</code>	302
JSON	<code>JsonResponse({...})</code>	200

Class Based View (CBV) — klass asosidagi view'lar

Hozirgacha **funksiya asosidagi view** (Function Based View, FBV) yozdik. Django yana bir usulni taklif qiladi — view'ni **klass** sifatida yozish (25-mavzudagi OOP!). Ko'p uchraydigan vazifalar uchun Django tayyor klasslar beradi va siz ulardan **meros** olasiz.

blog/views.py

```
from django.views import View
from django.views.generic import TemplateView

class SalomView(View):
    def get(self, request):                # GET so'rov uchun metod
        return HttpResponse("Salom, GET!")

    def post(self, request):               # POST so'rov uchun metod
        return HttpResponse("Salom, POST!")

class AloqaView(TemplateView):            # tayyor klass: shablonni
    ko'rsatadi                             # 35-mavzuda yaratamiz
    template_name = "blog/aloka.html"
```

blog/urls.py

```
path("salom/", views.SalomView.as_view(), name="salom"), # CBV uchun
.as_view()
```

	FBV (funksiya)	CBV (klass)
Yozilishi	Oddiy, aniq, boshlovchilar uchun tushunarli	Qisqa, lekin "sehr" ko'p
GET/POST ajratish	<code>if request.method == "POST":</code>	Alohida <code>get()</code> , <code>post()</code> metodlar
Qayta ishlatish	Qiyinroq	Meros orqali oson; tayyor <code>ListView</code> , <code>DetailView</code> , <code>CreateView</code> ...

Bu kursda asosan FBV ishlatamiz (tushunish oson), 38-mavzuda esa tayyor CBV'lar bilan CRUD qanchalik qisqa bo'lishini ko'rasiz.

Amaliy mashg'ulot

- 1 `blog/urls.py` yarating va `config/urls.py` da `include` qiling.
- 2 Yuqoridagi view'larni yozing va har bir manzilni brauzerda sinab ko'ring. `/maqola/abc/` ni ham oching — nima uchun 404? (`int` converter harflarni qabul qilmaydi.)
- 3 Bosh sahifadagi havolalarni `reverse("blog:maqola", args=[m["id"]])` bilan yasang.
- 4 Yangi yo'llar qo'shing:
 - `/kalkulyator/<int:a>/<int:b>/` — ikki sonning yig'indisi, ayirmasi, ko'paytmasini ko'rsatsin;
 - `/salom/<str:ism>/` — "Salom, Ali!" deb javob bersin;
 - `/api/maqolalar/` — JSON qaytarsin (`JsonResponse`).
- 5 `/tasodifiy/` — tasodifiy maqolaga `redirect` qilsin (`random.choice`).

Ko'p uchraydigan xatolar

- ❌ `Page not found (404)` — URL'da yakuniy `/` yo'q yoki yo'l `urlpatterns` da ro'yxatdan o'tmagan. Django xato sahifasida barcha mavjud yo'llar ro'yxatini ko'rsatadi — tekshiring.
- ❌ `maqola() got an unexpected keyword argument 'pk'` — URL'dagi nom (`<int:pk>`) va funktsiya parametri (`id`) mos emas.
- ❌ View `return` qilmaydi — `The view didn't return an HttpResponse object`.
- ❌ `NoReverseMatch` — `reverse` / `redirect` dagi URL nomi noto'g'ri yoki `app_name:` unutilgan.
- ❌ CBV'da `.as_view()` unutilgan.

O'zingizni tekshiring

? Django view'i nimani qabul qilib, nimani qaytaradi?

- A. HTML qabul qilib, SQL qaytaradi
- B. request (so'rov) qabul qilib, response (javob) qaytaradi
- C. URL qabul qilib, model qaytaradi
- D. Hech narsa qabul qilmaydi

? `path("maqola/<int:id>/", views.maqola)` — `/maqola/7/` so'ralganda view nima oladi?

- A. `id="7"` (matn)
- B. Hech narsa
- C. `id=7` (son)
- D. `maqola=7`

? Ilova URL'larini asosiy `urls.py` ga ulash uchun?

- A. `include("blog.urls")`
- B. `import blog`
- C. `connect("blog")`
- D. `urls.add(blog)`

? `/qidiruv/?q=futbol` dagi "futbol" ni view'da qanday olasiz?

- A. `request.POST["q"]`
- B. `request.q`
- C. `request.path`
- D. `request.GET.get("q")`

? Boshqa sahifaga qayta yo'naltiruvchi funksiya?

- A. `HttpResponse()`
- B. `redirect()`
- C. `Http404()`
- D. `goto()`

? Class Based View'ni URL'ga ulashda nima yoziladi?

- A. `views.SalomView`
- B. `views.SalomView()`
- C. `views.SalomView.as_view()`
- D. `views.SalomView.get`

Uyga vazifa

1. `pages` ilovasi (33-mavzu uyga vazifasi) uchun alohida `urls.py` yarating: `/haqida/`, `/aloqa/`, `/qoidalar/`.
2. `/jadval/<int:son>/` — berilgan sonning 1–10 gacha ko'paytirish jadvalini HTML jadval sifatida qaytaruvchi view.
3. `/yosh/<int:yil>/` — tug'ilgan yil bo'yicha yoshni hisoblovchi view; yil kelajakda bo'lsa — 404.

Yodda tuting

- View: `def view(request): return HttpResponse(...)`.
- Ilovada `urls.py` + `app_name`; asosiy faylda `path("", include("blog.urls"))`.
- Converterlar: `<int:id>`, `<str:nom>`, `<slug:nomi>` — view parametriga keladi.
- `request.GET.get("q")`, `raise Http404`, `redirect("app:nom")`, `JsonResponse`, `name=` + `reverse`.
- CBV: `class V(View): def get(self, request) → V.as_view()`.

☐ Darsni o'qib bo'ldim

✓ Test javoblari

1. Django view'i nimani qabul qilib, nimani qaytaradi? — **B) request (so'rov) qabul qilib, response (javob) qaytaradi**
2. `path("maqola/<int:id>/", views.maqola)` — `/maqola/7/` so'ralganda view nima oladi? — **C) id=7 (son)**
3. Ilova URL'larini asosiy `urls.py` ga ulash uchun? — **A) `include("blog.urls")`**
4. `/qidiruv/?q=futbol` dagi "futbol" ni view'da qanday olasiz? — **D) `request.GET.get("q")`**
5. Boshqa sahifaga qayta yo'naltiruvchi funksiya? — **B) `redirect()`**
6. Class Based View'ni URL'ga ulashda nima yoziladi? — **C) `views.SalomView.as_view()`**